

Weather station system

Connectivity in IoT

Albert Marquillas

September 13, 2026

Contents

1	Introduction	1
1.1	What is MQTT?	2
1.2	What is LoRa?	2
2	Objectives	2
3	General description of the weather station	2
3.1	Block diagram	3
3.2	Hardware design	3
3.3	Software architecture	4
4	Activity 1: MQTT communication with ESP32 and server	6
4.1	Objectives	6
4.2	Hardware Requirements	6
4.3	Software Setup	6
4.4	Guiding Questions	6
5	Practice 2: LoRa-based communication with MQTT integration	7
5.1	Objectives	7
5.2	Hardware Requirements	7
5.3	Software Implementation	7
5.4	Experiments on LoRa parameters	7
5.5	Guiding Questions	7
6	Conclusion	7

1 Introduction

The Internet of Things (IoT) enables devices to communicate over the internet, facilitating data collection and automation. Two key technologies for IoT communication are MQTT and LoRa. This document provides step-by-step instructions for two laboratory practices involving ESP32 microcontrollers, LoRa communication, and MQTT messaging.

1.1 What is MQTT?

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol designed for low-bandwidth, high-latency networks. It follows a publish-subscribe model where devices (clients) send data to an MQTT broker, which distributes messages to subscribers.

1.2 What is LoRa?

LoRa (Long Range) is a wireless communication technology designed for long-range, low-power transmission. It is widely used in IoT applications where devices need to communicate over large distances with minimal power consumption.

2 Objectives

1. **Develop skills in programming microcontrollers:** Gain experience in programming the ESP32 microcontroller using MicroPython with Thonny IDE.
2. **Implement and test LoRa and MQTT communication for data transmission:** Learn how to collect data from the sensors and transmit it using the LoRa and the MQTT protocol, enabling real-time monitoring and analysis of meteorological conditions.
3. **Enhance problem-solving and debugging skills:** Develop the ability to identify, diagnose, and resolve issues that arise during the development and testing of a weather station, focusing on both hardware and software debugging.
4. **Understand the practical applications and limitations of IoT weather stations:** Reflect on the practical implications of the IoT weather station, including its strengths, potential weaknesses, and areas for improvement, particularly in terms of accurate data collection, reliable communication, and responsive actuation.

These objectives aim to provide students with hands-on experience in IoT development, microcontroller programming, and system integration, all within the context of creating a functional weather station.

3 General description of the weather station

This scenario aims to develop a weather station designed to monitor and respond to various environmental conditions in real-time. It consists of two main components, both powered by ESP32 microcontrollers. The first component acts as the primary weather station, equipped with a suite of sensors to measure temperature, humidity, atmospheric pressure, CO2 levels, light intensity, sound levels, and wind speed. This station also includes a LoRa transceiver for long-range communication and an actuator, a 3-6V motor that spins a fan blade, used for automated environmental control based on the collected data.

The first station gathers meteorological data and transmits it to a second ESP32 module via the LoRa communication protocol. This second module is responsible for forwarding the data to a web server using the MQTT protocol, allowing the information to be sent to the Grafana cloud for storage, processing, and visualization. This cloud-based approach enables remote monitoring and data analysis, providing real-time insights into environmental conditions and

system performance.

The integration of sensors, actuators, and communication protocols allows the system to not only monitor environmental conditions but also respond autonomously to specific thresholds, making it a versatile and powerful tool for real-time weather monitoring and analysis.

3.1 Block diagram

The overall block diagram of the weather station is the one in the Figure 1.

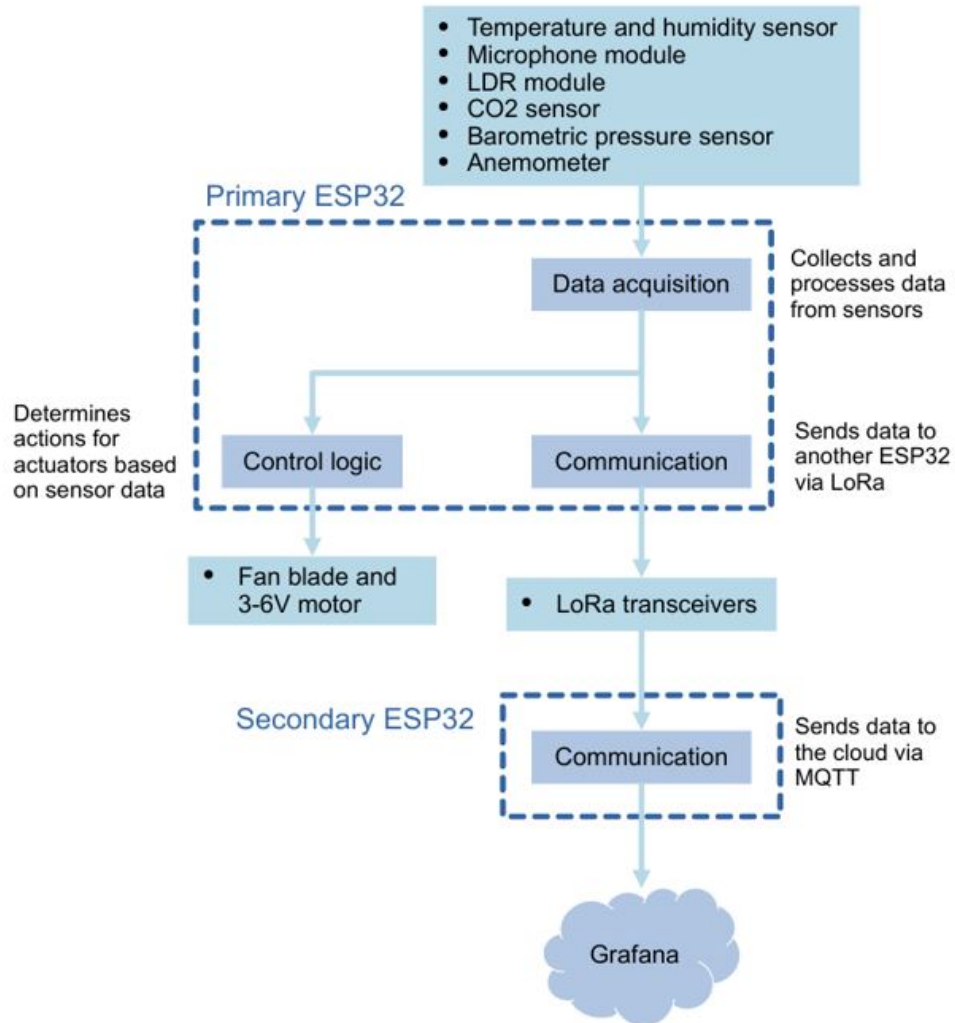


Figure 1: General block diagram.

3.2 Hardware design

The Figure 2 is a visual diagram that illustrates the connections and interactions between the different components (sensors, actuators, and microcontrollers). It provides a high-level overview of how data flows and control signals are managed within the system.

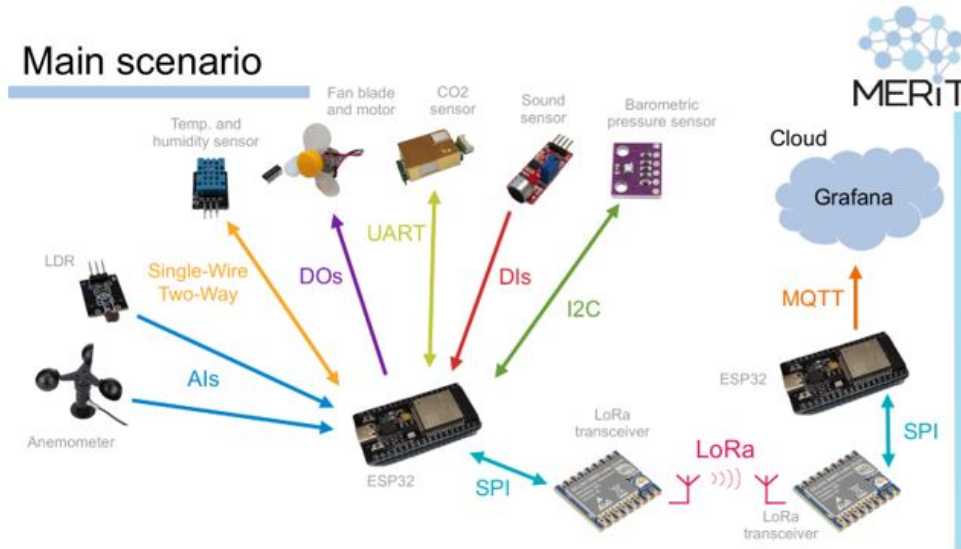


Figure 2: Connection diagram.

In the Figure 3 there are all the hardware connections that have to be done to build the weather station.

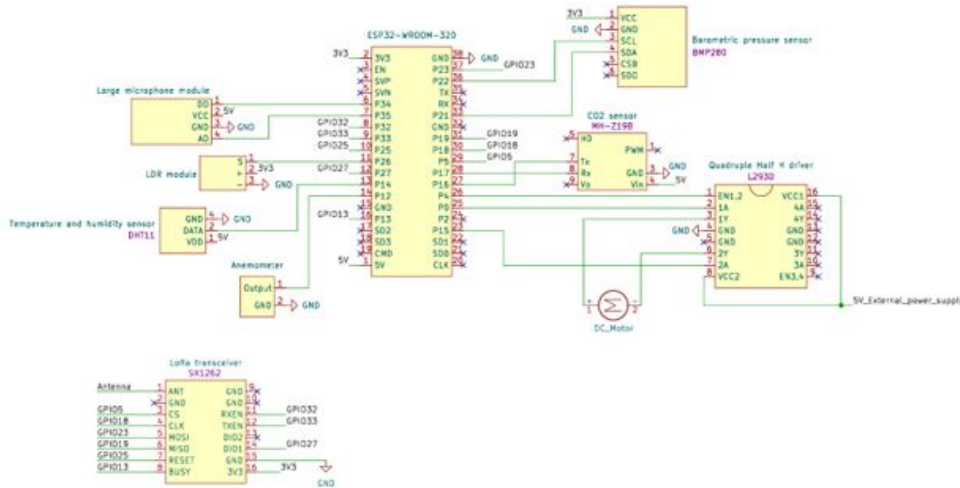


Figure 3: Connection diagram.

3.3 Software architecture

As it was described earlier, the system is formed by two ESP32s. One is in charge of recollecting the data from the sensors, controlling the fan motor, and sending the collected information via LoRa to the other ESP32. The second one receives this data and transmits it to the cloud via MQTT.

The first ESP32 continuously reads data from various sensors connected to it, including the temperature and humidity sensor, barometric pressure sensor, CO2 sensor, LDR, microphone

module, and anemometer. The temperature value read from the sensor is compared against a threshold to determine whether the fan blade should be activated. If the temperature exceeds the threshold, a signal is sent to the motor actuator to spin the fan. The first ESP32 processes the raw sensor data to format it appropriately for transmission. Basically, it converts analog readings to digital values and normalizes data. The processed data, now in a compact format, is transmitted from the first ESP32 to the second ESP32 using the LoRa communication protocol. The data is sent as a packet that includes all relevant sensor readings.

The second ESP32 establishes a connection with a Wi-Fi network to access the internet. Upon receiving the data via LoRa, it parses the incoming packet to extract the sensor readings. This microcontroller packages the sensor data into an MQTT message. The MQTT client, running on the ESP32, connects to an MQTT broker hosted on a web server. The ESP32 publishes the data to a specific MQTT topic, making it available for cloud-based applications or dashboards that subscribe to this topic.

The data published to the MQTT topic is received by a cloud server, where it can be stored in a database. The stored data is then used to generate real-time visualizations and alerts.

The general flowchart is the one seen in the Figure 4

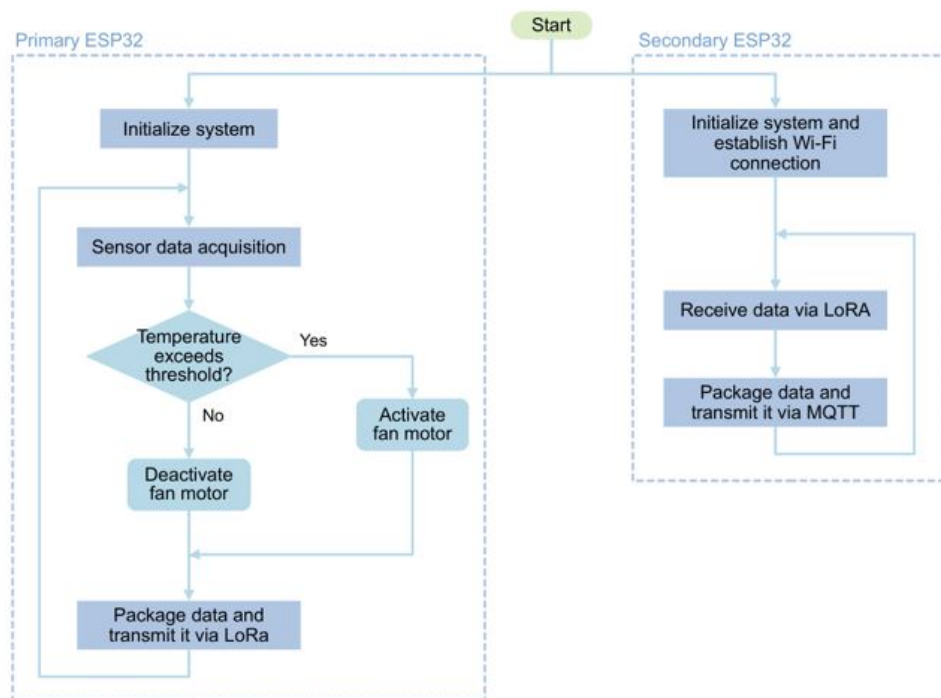


Figure 4: Software architecture.

4 Activity 1: MQTT communication with ESP32 and server

4.1 Objectives

The goal of this activity is to set up an ESP32-based sensor node that reads all the sensors data and sends the information to the server. To achieve this the data will be gathered from a DHT11 sensor, which provides the temperature and humidity. The information will have to be published to an MQTT broker running in a Docker container with Mosquitto.

4.2 Hardware Requirements

- ESP32 development board (e.g., Heltec LoRa 32 V2)
- DHT11 temperature and humidity sensor
- USB cable for programming
- Computer with Docker installed

4.3 Software Setup

1. Install Mosquitto on Docker
2. Install the Arduino IDE for programing ¹ and necessary libraries:
 - PubSubClient for MQTT communication - recommended: *PubSubClient by Nick O'Leary v2.8*
 - DHT for sensor readings - recommended: *DHT sensor library by Adafruit v1.4.6*
 - WiFi for network connectivity - recommended: *Heltec ESP32 Dev-Boards v2.1.2*
3. Write an ESP32 script in C++ to:
 - Read temperature and humidity data from the DHT11 sensor.
 - Connect to a Wi-Fi network.
 - Publish sensor readings to a specified MQTT topic. The information should be transmitted in json formatting.
4. Develop a subscriber application:
 - Using a tool of the students preference, plot temperature and humidity values in real-time. Some suggestions are using Python with Matplotlib or using a HTML framework to create a more realistic interface for presenting the live data.

4.4 Guiding Questions

- Why is MQTT suitable for IoT applications?
- How does the publish-subscribe model improve efficiency over a request-response model?
- What are the advantages of running Mosquitto in a Docker container?

¹Arduino IDE and C++ programming are not mandatory for doing this activity though it is very recomendable, alternatives may be using MicroPython or pure C.

5 Practice 2: LoRa-based communication with MQTT integration

5.1 Objectives

This activity extends the first one by introducing LoRa communication between two ESP32 modules. The first ESP32 reads the DHT11 sensor from before and sends data via LoRa to the second ESP32, which then publishes the data to the MQTT broker.

5.2 Hardware Requirements

Same as activity 1, but with an additional ESP32 for receiving LoRa messages.

5.3 Software Implementation

1. Install the Arduino LoRa library.
2. The first ESP32:
 - Reads temperature and humidity data.
 - Transmits data using the LoRa module.
3. The second ESP32:
 - Receives LoRa messages.
 - Forwards the received data to the MQTT broker.
4. The subscriber application remains unchanged and visualizes data in real-time.

5.4 Experiments on LoRa parameters

To optimize communication, students should experiment with the following:

- Adjusting the bandwidth and data rate to balance transmission speed and range.
- Changing transmission power to evaluate energy consumption and signal strength.
- Measuring Quality of Service (QoS) by evaluating packet loss over different distances.

5.5 Guiding Questions

- How does LoRa compare to Wi-Fi and Bluetooth in terms of range and power consumption?
- What are the trade-offs of using lower data rates in LoRa transmission?
- How does increasing transmission power affect battery life and interference?

6 Conclusion

These activities provide hands-on experience with LoRa and MQTT, essential protocols for general IoT applications. By implementing and optimizing these communication techniques, students will gain valuable insights into real-world IoT deployments and data transmission strategies.